SemanticLog: Towards Effective and Efficient Large-Scale Semantic Log Parsing

Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu, Jihong Guan, Qi Zhou, and Shuigeng Zhou, *Senior Member, IEEE*

***Abstract*—Logs of large-scale cloud systems record diverse system events, ranging from routine statuses to critical errors. As the fundamental step of automated log analysis, log parsing is to transform unstructured logs into structured data for easier management and analysis. However, existing syntax-based and deep learning-based parsers struggle with complex real-world logs. Recent parsers based on large language models (LLMs) achieve higher accuracy, but they typically rely on online APIs (e.g., ChatGPT), raising privacy concerns and suffering from network latency. Moreover, with the rise of artificial intelligence for IT operations (AIOps), traditional parsers that focus on syntax-level templates fail to capture the semantics of dynamic log parameters, limiting their usefulness for downstream tasks. These challenges highlight the need for semantic log parsing that goes beyond template extraction to understand parameter semantics. This paper presents SemanticLog, an effective and efficient semantic log parser powered by open-source LLMs. SemanticLog adapts the structure of LLMs to the log parsing task, leveraging their rich knowledge while safeguarding log data privacy. It first extracts informative feature representations from log data, then refines them through fine-grained semantic perception to enable accurate template and parameter extraction together with semantic category prediction. To boost scalability, SemanticLog introduces the EffiParsing tree for faster inference on large-scale logs. Extensive experiments on the LogHub-2.0 dataset show that SemanticLog significantly outperforms the state-of-the-art log parsers in terms of accuracy. Moreover, it also surpasses existing LLM-based parsers in efficiency while showcasing advanced semantic parsing capability. Notably, SemanticLog employs much smaller open-source LLMs compared to existing LLM-based parsers (mainly based on ChatGPT), while maintaining better capability of log data privacy protection.**

***Index Terms*—Semantic log parsing, log analysis, large language models.**

Received 18 January 2025; revised 8 October 2025; accepted 21 October 2025. Date of publication 24 October 2025; date of current version 9 January 2026. The work of Jihong Guan was supported by National Natural Science Foundation of China (NSFC) under Grant 62172300 and Grant 62372326. Recommended for acceptance by D. Bianculli. *(Corresponding authors: Shuigeng Zhou; Jihong Guan.)*

Chenbo Zhang, Wenying Xu, Lu Zhang, and Shuigeng Zhou are with Shanghai Key Lab of Intelligent Information Processing, and the School of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200438, China (e-mail: cbzhang21@m.fudan.edu.cn; sgzhou@fudan.edu.cn).

Jinbu Liu, Guiyang Liu, and Qi Zhou are with Alibaba Group, Hangzhou 311121, China.

Jihong Guan is with the School of Computer Science and Technology, Tongji University, Shanghai 201804, China (e-mail: jhguan@tongji.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3625121

I. Introduction

LOGS are a vital source of information reflecting the running statuses of modern cloud systems. Engineers can use logs to diagnose and debug systems effectively. With the rapid development of cloud systems, logs in real scenarios have become increasingly complex and voluminous [1], [2], [3], [4]. Manually analyzing these logs is inefficient and impractical, making log parsing a critical task in automated log analysis.

The goal of log parsing is to transform unstructured or semi-structured raw logs into structured formats, which can then be used for further downstream management, understanding, and analysis tasks [5], [6], [7], [8]. As shown in Fig. 1, a raw log consists of a log header (e.g. "Timestamp" and "Level") and log content. The log header has a relatively fixed format, which can be separated from the log content using methods like regular expressions. The log parsers extract static log templates from the log content, and replace dynamic log parameters with the wildcard ``<*>''.

Existing log parsers can be mainly divided into three types: syntax-based, deep learning-based, and LLM-based. Syntax-based log parsers [4], [9], [10], [11], [12], [13] use strategies like frequent pattern mining, log clustering, and heuristic methods to extract the invariant parts of logs as templates. Deep learning-based log parsers [14], [15], [16], [17] employ natural language processing (NLP) techniques to train models with annotated logs, distinguishing between log templates and parameters. However, the generalization of these methods is relatively poor. As the logs evolve, it is needed to adjust a large number of hyperparameters or retrain the model. With the rapid advancement and great success in NLP of large language models (LLMs) [18], [19], [20], [21], [22], LLMs are increasingly used for log parsing. However, most existing LLM-based log parsers [23], [24], [25], [26] rely on OpenAI's online API, which is highly affected by network latency fluctuations and ChatGPT version changes, thus inefficient and expensive. Furthermore, as log data often contains a significant amount of private business information, the use of online API services incurs privacy concerns. Therefore, they are more suitable for offline processing locally.

On the other hand, with the popularity of artificial intelligence for IT operations (AIOPs), traditional log parsers can no longer meet the needs of many downstream tasks such as anomaly detection [5], [6], [7], [8], root cause analysis [27], [28], [29], and log summarization [30], [31], [32]. This is

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

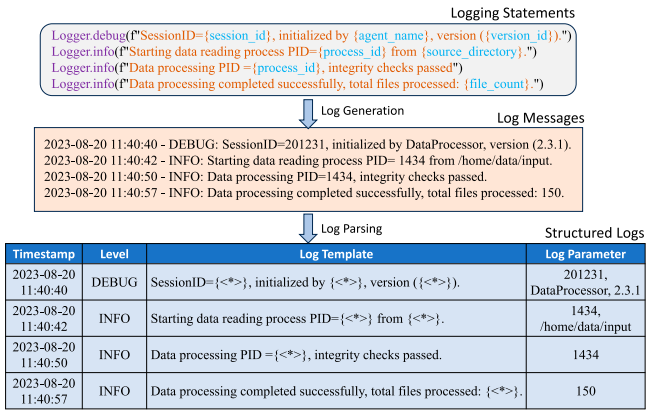

Fig. 1. An example to illustrate the log parsing process.

because they focus on extracting log templates while overlooking the rich semantic information contained in log variables. Taking the log ``Connection completed. Connection status: 0'' as an example, traditional log parsers cannot understand the semantics of log parameters. They only extract the log template ``Connection completed. Connection status: <*>'' and treat the log parameter ``0'' as a regular number for subsequent anomaly detection. However, in addition to extracting the log templates and parameters, semantic log parsers try to accurately understand that the parameter ``0'' is a discrete status code. This helps the downstream tasks to use targeted anomaly detection algorithms, thus improving anomaly detection performance.

Therefore, we think that the new generation of log parsers should feature in: (1) being effective in parsing logs from diverse sources, addressing the limitations of existing methods in performance and log privacy protection. (2) being capable of semantic log parsing [16], [17], [33], i.e., predicting the semantics of log parameters while extracting log templates, and outputting both log templates and semantic labels of log parameters for downstream AIOps tasks. (3) being efficient in parsing massive real-time logs from large-scale cloud systems.

To meet these requirements above, in this paper we present **SemanticLog**, a novel, accurate, and rapid semantic log parser, which is composed of three major modules: **LogLLM**, **SemPerception module**, and **EffiParsing tree**. Each module is designed to target a specific requirement mentioned above. Specifically, LogLLM serves as our model's backbone, adapting the structure of open-source LLMs to make LLM suitable for log parsing. This design leverages the rich pre-trained knowledge within LLM to effectively process logs from various sources, consequently enhancing model generalization, while protecting log privacy because the logs are processed locally. The SemPerception module, while preserving complete log token information, employs multi-head cross-attention [34] to fully perceive the semantics of log tokens. This allows the model to accurately predict semantic labels of log parameters while extracting log templates. The EffiParsing tree uses a proposed prefix tree structure to store log templates and match predictions for new logs, significantly improving the model's efficiency in parsing massive logs.

We conduct extensive experiments on the public benchmark LogHub-2.0 [35], which show that SemanticLog achieves the best results on all metrics on average for both traditional log parsing and semantic log parsing tasks. SemanticLog also demonstrates excellent robustness across different log sources and outperforms all LLM-based parsers, deep learning-based parsers, and some syntax-based parsers in efficiency. Importantly, SemanticLog is an LLM-based log parser that can run offline while ensuring the privacy of log data.

Our contributions are as follows: 1) We analyze the limitations of existing log parsers and identify three key requirements for the new generation of log parsers suitable for real-world applications, highlighting the importance of semantic log parsing for cloud systems and AIOps tasks. 2) We develop **SemanticLog**, a new, effective and efficient LLM-empowered semantic log parser, which leverages the comprehensive pre-trained knowledge of LLMs and well-designed modules to achieve accurate and rapid semantic parsing of large-scale logs. 3) We evaluate SemanticLog on the public benchmark LogHub-2.0. Experimental results show that SemanticLog achieves SOTA performance in both traditional log parsing and semantic log parsing tasks.

The rest of this paper is organized as follows: Sec. II details the background and motivation. Sec. III presents the log semantic category hierarchy. Sec. IV provides the problem statement for semantic log parsing. Sec. V introduces the method. Sec. VI and VII are experimental design and results. Sec. VIII discusses some related issues. Sec. IX, X and XI explore validity threats and related work, and conclude the paper.

II. Background and Motivation

A. Evolution of Log Parsing

Log parsing, serving as the critical task in automated log analysis, has garnered wide research interest from both industry and academia [3]. Researchers have been devoted to extracting static log templates and dynamic log parameters from raw logs, thereby converting unstructured or semi-structured raw logs into structured forms. This transformation facilitates further log analysis of these structured data. However, with the wide deployment of AIOps, this relatively simple routine increasingly fails to meet the demands of complex downstream tasks such as anomaly detection [5], [6], [7], [8], root cause analysis [27], [28], [29], and log summarization [30], [31], [32]. These drawbacks stem from the fact that log parameters, carrying crucial information including timestamps and location details, etc., are often represented merely by placeholders, e.g. ``<*>'', which ignore the semantics of log parameters, leading to a significant loss of original log information.

As a result, the growing complexity of downstream tasks in log analysis has led to an evolution in the requirements for log parsing. It is no longer sufficient to merely extract log templates and parameters from raw logs. Instead, there is a need to interpret the semantics of these log parameters to enable more advanced analysis in downstream tasks. This marks the

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

shift from traditional log parsing to semantic log parsing. Such an advancement would significantly enhance AIOps, particularly in complex log analysis scenarios that demand a deep understanding of vast amounts of log data. Recent studies [17], [33] have shown that by extracting more valuable semantics from raw logs, semantic log parsing can significantly boost the performance of downstream tasks.

B. Motivation

With the advance of LLMs and their applications in software engineering, researchers have already utilized LLMs for log parsing. Although LLM-based parsers have shown promising potential, they still fail to meet the demands of real-world applications, particularly in performing semantic log parsing. Upon analysis, we believe that the new generation of LLM-based parsers should have the following features:

Effectiveness with log privacy protection. Effectiveness is the primary performance requirement for log parsers, encompassing accuracy and robustness in log parsing. While LLMs contain comprehensive pre-trained knowledge, they are designed for text generation rather than log parsing. Le et al. [23] demonstrated that directly using LLMs to generate log templates leads to poor results. Subsequent works [24], [25], [36] have largely adopted in-context learning, providing examples to LLMs to help them better understand log parsing tasks and improve effectiveness. Despite the promising performance of the latest LLM-based parsers, a critical issue remains: these methods rely on OpenAI's ChatGPT API service. Since logs often contain sensitive information, companies are typically prohibited from uploading them to the internet to avoid privacy risks. The privacy concern makes existing LLM-based parsers difficult to be applied in practice. Therefore, achieving effective log parsing while safeguarding log privacy is a crucial practical requirement.

Semantic log parsing capability. LLMs, with their powerful text comprehension and generalization capabilities, are inherently well-suited for semantic log parsing. Their abilities to understand text semantics position them as promising tools for interpreting the semantics of log parameters, which is crucial for advanced log analysis tasks. However, despite this potential, existing LLM-based parsers are ineffective in performing semantic log parsing due to the lack of targeted design. Current methods focus on extracting log templates and parameters but fail to capture the semantic information behind key log parameters like timestamps and error codes. This limits the usage of LLM-based parsers in the real world. Therefore, enabling LLMs to interpret log parameter semantics accurately is a vital practical requirement.

High efficiency for massive logs. In real-world scenarios, the volume of logs is enormous. For example, Alibaba Inc.'s cloud computing systems generate approximately 30 to 50GB of logs per hour [37]. Therefore, efficiency is a critical performance requirement for log parsers. While LLMs possess powerful capabilities, they are hindered by slow processing speeds, making the direct use of LLMs for log parsing impractical. Although existing approaches have attempted to reduce the inference burden of LLMs by utilizing caching mechanisms, these methods rely on online API services and are constrained by network latency, resulting in low and unstable efficiency. Thus, achieving stable and high-efficiency parsing for massive logs is a crucial practical requirement.

In response to the requirements outlined above, this paper introduces **SemanticLog**, the first semantic log parser based on open-source LLMs. First, to address effectiveness and privacy concerns, SemanticLog uses relatively small open-source LLMs that can run offline, rather than large online LLMs. By modifying the internal architecture of LLMs, we develop the LogLLM module. Unlike traditional generative LLMs, LogLLM retains rich pre-trained knowledge while utilizing bidirectional attention to better suit log parsing tasks. Second, for semantic log parsing, SemanticLog incorporates the Sem-Perception module (SPM), which aggregates the features of tokenized log data with learnable semantic vectors and maps them into a fine-grained semantic space. This allows it to not only distinguish between log templates and parameters but also perceive the semantics of log parameters. Third, to lift efficiency, SemanticLog employs the EffiParsing tree, a prefix tree structure used to store previously parsed templates. When new logs arrive, it performs rapid matching and retrieval, avoiding redundant predictions for the same log templates and significantly speeding up the parsing process. Overall, SemanticLog achieves a balance between effectiveness, efficiency, and privacy protection, making it a promising solution for semantic log parsing.

III. Fine-Grained Log Semantic Category System

A proper log semantic category system is the foundation of effective semantic log parsing. VALB, through a manual review of the LogHub dataset, defines a 10-category log semantic system. However, we find that these categories are too coarse for practical application. Revisiting VALB's system, we apply it to annotate the more diverse LogHub-2.0 benchmark dataset. During this process, we observe that some log templates cannot be accurately annotated due to the limitations of the system. For instance, the system fails to specifically categorize frequently appearing and critical exception information in logs, instead grouping them under the "Others" category, which results in the loss of valuable semantic information.

To address this issue, we thoroughly review the LogHub-2.0 dataset and identify logs that are difficult to categorize using the VALB semantic system. Our team consists of two subgroups, each containing two experienced engineers and two PhD students. These subgroups independently define new semantic categories for the selected logs and annotate them accordingly. After completing the annotation process, we compare the results from both subgroups. Annotations that are consistent between the two subgroups are considered correct, while discrepancies are discussed and related logs are re-annotated until a consensus is reached.

Through this process, we refine the original VALB log semantic system and expand it to contain 16 fine-grained semantic

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FINE-GRAINED LOG SEMANTIC CATEGORY SYSTEM

Category Name	Abbrev.	Description	Example	Proportion
Object ID	OID	Identification information of an object.	Adding an already existing block **blk_-2074647664485597823**	28.9%
Object Name	OBN	Name of an object.	setting hostname to "**authorMacBook-Pro.local**"	13.6%
Location Indicator	LOI	Location information of an object.	Client attempting to establish new session at **/10.10.34.25:46535**	18.4%
Type Indicator	TID	Type information of an object or an action.	setDeviceClassType=**1**	0.2%
Status Indicator	SID	Status code of an object or an action.	needAutoSync autoSyncSwitch is **open**	4.0%
Time Point of Action	TPA	Specific time an action occurs.	Setting clock (localtime): **Wed Dec 21 13:07:25 MST 2005** succeeded	0.3%
Time Duration of Action	TDA	Time duration of an action.	Scheduled snapshot period at **10** second(s).	9.2%
Object Amount	OBA	Amount of objects.	Started **6** remote fetches	3.9%
Computing Resources	CRS	Information of computing resources.	MemoryStore started with capacity **17.7 GB**	10.6%
Data Content	DCT	Data content in logs.	pan_kmod: registered subdev 0x78 ("**PANASAS kernel trace facility**")	0.6%
Error and Exception	ERR	Information of errors and exceptions.	fail errorCode = 4, errorMessage = **ERR_DATA_INSERT**	2.0%
Command	CMD	Information of executed commands.	(root) CMD (**run-parts /etc/cron.hourly**)	1.1%
Object Order	OBO	Order information of an object.	Read completed tasks from history **10**	5.0%
Interval	INR	Information of data intervals.	Copyright (c) **1999-2005** LSI Logic Corporation	0.1%
Version Information	VER	Details about the version of an object.	Version **1.0.6** Starting	0.6%
Other Parameters	OTP	Other information.	Reading broadcast variable **4506** took 6 ms	1.5%

categories. As presented in Table I, these categories include Object ID (*OID*), Object Name (*OBN*), Location Indicator (*LOI*), Type Indicator (*TID*), Status Indicator (*SID*), Time Point of Action (*TPA*), Time Duration of Action (*TDA*), Object Amount (*OBA*), Computing Resources (*CRS*), Data Content (*DCT*), Error and Exception (*ERR*), Command (*CMD*), Object Order (*OBO*), Interval (*INR*), Version Information (*VER*), and Other Parameters (*OTP*). Compared to the coarse-grained categories in VALB, our refined category system introduces more nuanced semantics. For example, it distinguishes between TDA and TPA, which are previously grouped together. Additionally, it includes important categories such as ERR and OBO, which do not exist in the original system. We validate this fine-grained semantic system on the company's business logs, and 96% of log parameters can be accurately defined. This fine-grained semantic system is currently deployed on Alibaba Cloud and has been running stably for six months.

IV. PROBLEM STATEMENT

The goal of semantic log parsing is to extract log templates from raw logs and to predict the semantic meaning implied in the log parameters. We model this task as a log token classification problem. Concretely, consider a raw log T that, after tokenization, is divided into n log tokens t_i, that is, $T = \{t_1, t_2, t_3, \ldots, t_n\}$. Our method, SemanticLog, is designed to distinguish between template tokens and parameter tokens within each log, and to predict the semantic labels of parameter tokens. Formally, the output of the model $\mathcal{M}$ can be represented as $Y = \{y_1, y_2, y_3, \ldots, y_n\}$, and we have

$$y_i = \mathcal{M}(t_i) = \begin{cases} O & \text{if } t_i \textit{ is a template token} \\ S_i & \text{if } t_i \textit{ is a parameter token} \end{cases} \tag{1}$$

where O is the label for template token, S_i represents the set of predicted semantic labels for parameter token t_i, which here consists of a certain number (3 in this paper) of labels with the highest prediction scores (e.g. $S_i = [OID, OBN, OTP]$). Finally, for log tokens with the predicted label O, we convert their prediction result back to the original token. For log tokens with the predicted label S_i, we keep their prediction result as S_i. The reason why we define a set of three semantic labels, instead of a single semantic label as the prediction result is that in many cases the semantic label of a log parameter may be relatively vague, and is related to multiple categories. For instance, the log token `''Mike001''` may elicit divergent opinions among software engineers, with some interpreting it as an Object ID (OID) and others as an Object Name (OBN). This variability requires the model to predict a ranked list of the most relevant semantic categories, sorted by their likelihood. Consequently, engineers are afforded the flexibility to select the semantic labels that are most closely relevant to their professional judgment from the predicted label list, thereby accommodating various perspectives in the domain.

Fig. 2 illustrates the difference between traditional log parsing and semantic log parsing. Our model can effectively understand the logs. In addition to extracting log templates from raw logs, it also predicts the semantic categories of log parameters. This provides great support for software engineers and downstream tasks in AIOps.

V. METHODOLOGY

In this section, we first present an overview of our proposed method **SemanticLog** in Sec. V-A. Then, in Sec. V-B, we provide a detailed introduction to the architecture and major components of SemanticLog.

A. Overview

In real-world systems, log data originate from diverse sources, and their templates evolve frequently over time. Although existing log parsers perform well on benchmark datasets, they often fail in practice due to their reliance on either

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

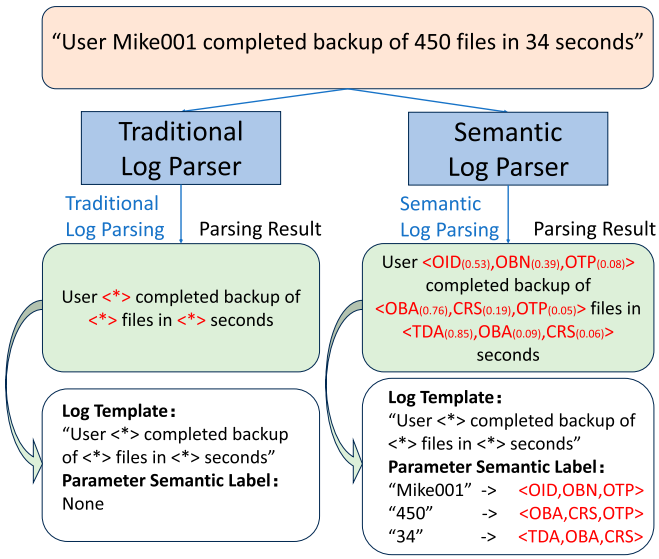

Fig. 2. Examples of traditional log parsing and semantic log parsing.

manually defined rules [4], [9], [10], [11], [12], [13] or supervised deep learning models [14], [16], [17] that are sensitive to data distribution shifts. These approaches typically require frequent manual updates or retraining, leading to significant maintenance overhead. UniParser [15], the first parser designed for heterogeneous logs, still struggles with low accuracy and poor generalization due to limitations in its model structure.

To address these challenges, we propose **SemanticLog**, a semantic log parsing framework designed to handle heterogeneous log data efficiently and accurately. By leveraging parameter-efficient fine-tuning (PEFT) [38] on LLMs, SemanticLog aligns the pre-trained knowledge of the model with the requirements of semantic log parsing. In addition to conventional log parsing, it can identify the semantic categories of log parameters, enabling semantic log parsing. This design endows SemanticLog with strong generalization power, allowing it to handle unseen log formats without retraining.

The architecture of SemanticLog consists of three core components: **LogLLM**, the **SemPerception module**, and the **EffiParsing tree**, which work collaboratively to achieve accurate, efficient, and generalizable log parsing. Concretely, the core of SemanticLog is **LogLLM**, an adapted version of open-source LLMs that treats log parsing as a token classification task rather than a text generation task. This design enables LogLLM to fully capture contextual information by removing the causal mask commonly used in LLMs, thereby enhancing feature extraction for each log token. To further improve semantic understanding, SemanticLog introduces the **SemPerception module (SPM)**. SPM leverages multi-head cross attention to aggregate subword features into a unified representation, ensuring that the semantics of each log token are comprehensively captured and effectively classified. Finally, to support efficient log parsing in real scenarios, SemanticLog incorporates the **EffiParsing tree**, a prefix-tree-based caching mechanism inspired by Drain. It organizes log templates efficiently and accelerates parsing speed by reusing previously parsed results based on similarity matching.

These components are trained offline using annotated logs from multiple sources through PEFT, following the data-splitting protocol of UniParser to ensure that training and test log sources are mutually exclusive. As a result, SemanticLog can perform semantic log parsing on unseen log sources without further training. Compared with prior LLM-based methods [24], [25], [36], [39], SemanticLog achieves more stable and accurate parsing while preserving log privacy and avoiding dependence on costly online APIs.

B. Model Architecture and Major Components

As shown in Fig. 3, SemanticLog consists of three major components, **LogLLM**, **SemPerception module**, and **EffiParsing tree**, which are responsible for extracting log features, perceiving log semantics, and accelerating the parsing process on large-scale log data. In the following subsections, we introduce the three components in detail.

1) LogLLM: LLMs have been increasingly used in log parsing. Most existing LLM-based log parsers such as DivLog [25], LILAC [24], and LUNAR [36] employ In-Context Learning [40] to leverage ChatGPT for log parsing. An alternative approach, exemplified by LLMParser [26], utilizes open-source LLMs like ChatGLM [41] and LLaMA [21], performs instruction supervision fine-tuning for log parsing. Despite these advancements, concern persists regarding the log privacy, and the outcome achieved so far is not satisfactory.

The reason for the problems above lies in that these methods [24], [25], [26], [36] ignore the characteristics of the log parsing task. Specifically, since LLMs are designed for text generation tasks, these methods naturally treat log parsing as a text generation task. They try to obtain log templates by inputting specific instructions and original logs as prompts to LLMs. However, the output often includes errors such as incomplete parameter recognition, the inclusion of extraneous text, and misalignment between the template and the log structure. These inaccuracies stem from the inherent uncertainty in LLM's output, which, although somewhat manageable through adjustments in model "temperature," cannot be entirely solved. Such errors might be negligible in general text generation tasks, but in log parsing — the first step of automated log analysis, accuracy is crucial. Even minor deviations can cause many important log parameters to be missed, significantly affecting downstream tasks like anomaly detection. Therefore, text generation methods may be not suitable for log parsing, indicating that existing methods directly applying LLMs for log parsing are inappropriate and inefficient.

The problem becomes especially evident with smaller open-source LLMs, such as T5 and LLaMA2-7B. LLM-based parsers (e.g., LLMParser [26]) built on these models perform worse than deep learning-based parsers [14], [17] with significantly fewer parameters. While the powerful capabilities of models like ChatGPT can obscure this issue to some extent in online LLM-based parsers (e.g., LILAC [24], LUNAR [36]), they face practical limitations due to log privacy concern and instability

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

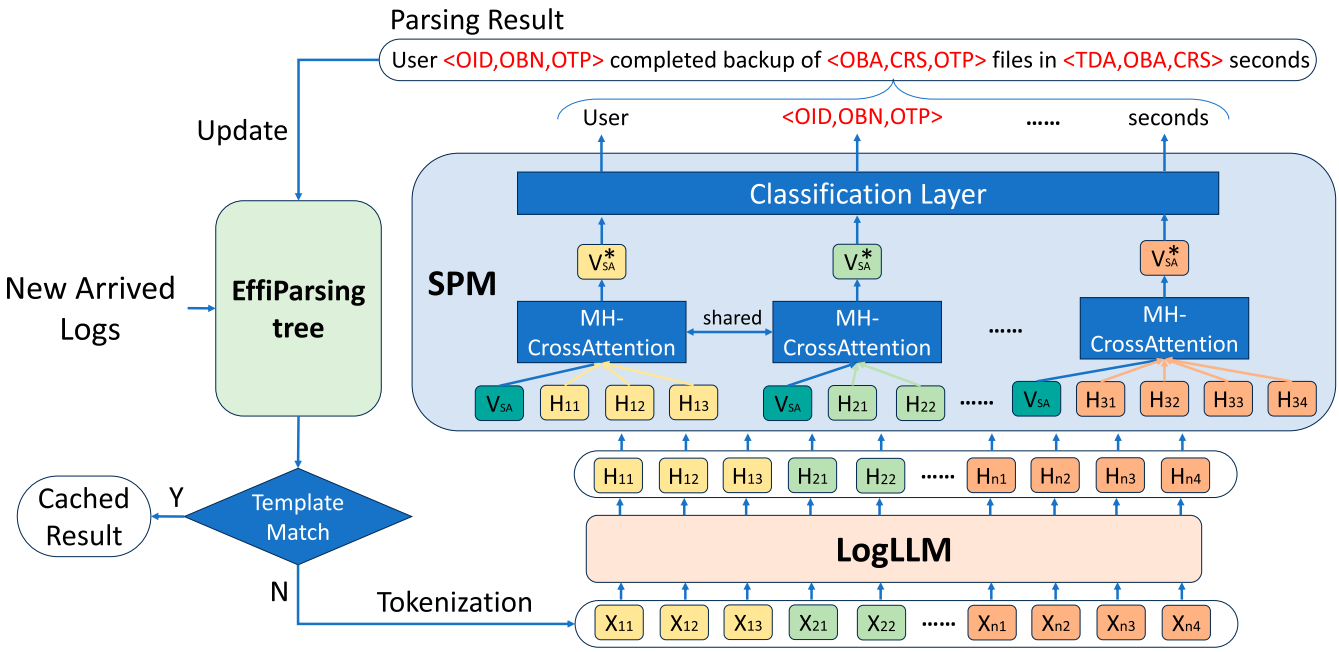

Fig. 3. The architecture of SemanticLog.

from network latency, making them undesirable for real-world applications.

Nevertheless, the success of LLMs across various domains indicates that even smaller open-source models have significant potential due to their pre-trained knowledge. The poor performance of current methods stems from the fact that they do not adapt LLMs to the specific characteristics of log parsing tasks, but instead treat log parsing as a text generation task. To effectively use LLMs for semantic log parsing, we believe a fundamental shift of methodology is necessary. We propose to cast log parsing as a token classification task. While LLMs excel in most NLP tasks, studies [42] have shown that directly applying instruction-tuning for token classification with LLMs yields suboptimal results. This limitation is attributed to the intrinsic autoregressive structure of LLMs. In log parsing, this problem also exists. LLMs typically employ the decoder-only autoregressive architecture, where a causal mask is utilized in the decoder blocks to prevent information leakage during text generation. This mechanism restricts the model to access only unidirectional information, thereby limiting its ability to perceive the full context. However, the classification of log tokens relies not just on the tokens but also on their contexts. For instance, special delimiters (e.g. ``()'', ``='', ``+'') and specific tokens (e.g. ``to'', ``from'' and ``on'') are often observed adjacent to log parameters, and are semantically relevant. The causal masking consequently hinders the model's ability to capture such crucial contextual information necessary for accurate log token classification.

To cure this limitation, we design **LogLLM**. As illustrated in Fig. 4(a), LogLLM modifies the architecture of open-source LLMs(e.g., LLaMA [22]) by removing the causal mask, enabling LogLLM to perceive the entire context of the log when extracting features of the current log token. Unlike existing LLM-based parsers, LogLLM treats log parsing as a token classification task rather than a text generation task. This adjustment allows LogLLM to fully leverage the context of log tokens to extract more accurate latent vector representations of log features, which are then fed into the SemPerception Module for feature fusion and semantic perception. The details are delayed to Sec. V-B2. Our design maximizes the capabilities of LLMs, enabling SemanticLog to outperform ChatGPT-based LLM parsers even when using much smaller open-source LLMs with fewer parameters, while preserving the privacy of log data. Consequently, LogLLM ensures a coherent and comprehensive understanding of logs, significantly enhancing LLMs' performance in log parsing.

2) SemPerception Module: In ordinary token classification tasks, a common approach is to tokenize the input into a series of subwords, and then use only the first subword for model training. Taking ``Apple'' as an example, it will be tokenized into subwords ``App'' and ``##le'', and only the first subword is forwarded to the model for feature extraction and classification. This subword's classification output is then considered as the entire token's label. This approach is sufficient for standard NLP tasks, where tokens are usually short common words whose semantics can be adequately captured by initial subwords, and for some common words, the tokenizer will not even tokenize them. However, this approach is not suitable for log parsing, where log tokens often extend well beyond the length of standard words, as in the cases of file paths (e.g. ``/home/tmp/Project/SourceCode/demo.py''), URLs (e.g. ``https://www.google.com''), or user-defined identifiers (e.g. ``MyFunctionForDebug01''). Using only the first subword leads to a loss of

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

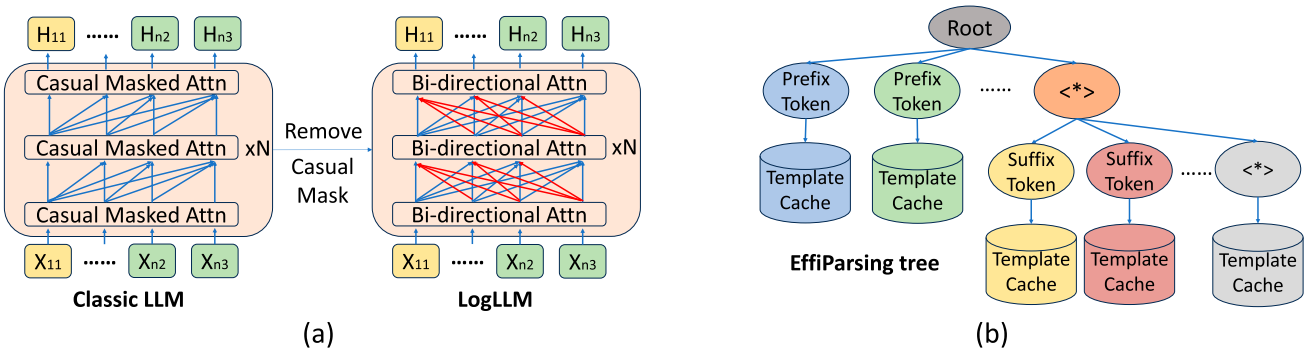

Fig. 4. (a) Comparison between LogLLM and classic LLM; (b) the EffiParsing tree structure.

semantics, detrimental to the subsequent semantic log parsing.

To address this issue, we propose the **SemPerception module** (**SPM** in short). Recognizing that each subword carries a piece of the overall semantics, as shown in Fig. 3, SPM employs the multi-head cross attention [34] mechanism to aggregate subword features into a learnable vector V_{SA}. Concretely, for the i-th log token t_i, the corresponding aggregated vector V_{SA}^* is calculated by

$$V_{SA}^* = Attn\Big(V_{SA}W_Q, [H_{i1}, H_{i2}, \cdots, H_{in}]W_K, [H_{i1}, H_{i2}, \cdots, H_{in}]W_V\Big) \quad (2)$$

where $Attn$ is the standard multi-head cross attention [34], H_i represents the subword features of t_i. By aggregating the features of various subwords, V_{SA}^* captures the semantics of log tokens, which is crucial for lengthy and complex log tokens where semantic meaning is not readily perceptible when considering subwords separately. The shared weights between attention modules are adopted to enhance the model's capacity for transfer learning and pattern recognition across diverse log tokens. V_{SA} serves as the linchpin in this architecture, synthesizing the subword features into a coherent whole. As training proceeds, the learnability of V_{SA} enables the model to continuously optimize it based on the training logs, making it increasingly suitable for semantic log parsing.

Finally, V_{SA}^* is fed into a classification layer for log semantic classification, and the loss function is as follows:

$$Loss = -\sum_{i=1}^{K} y_i log\left(\frac{e^{FC(V_{SA}^*)_i}}{\sum_{k=1}^{K} e^{FC(V_{SA}^*)_k}}\right) \quad (3)$$

where y_i is the semantic label of the target log token, FC denotes the fully connected layer classifier, and K is the total number of log semantic categories, encompassing all parameter and template classes. During training, we calculate the loss for each log token. In the inference stage, based on the predictions from the classifier, for log tokens predicted as parameters (where the maximum predicted probability category belongs to parameters), we retain the top three parameter categories with the highest prediction probabilities as a result. For log tokens predicted as templates, we keep the template category as the prediction result.

3) EffiParsing Tree: In real scenarios, cloud systems generate tens of thousands of logs per second. While existing deep learning-based parsers [14], [15], [16], [17] have surpassed syntax-based parsers [10], [11] in parsing accuracy, their low running efficiency limits their application in real-world scenarios. LLM-based log parsers [23], [24], [25], [26] face the same challenge: despite LLMs' powerful capabilities and good generalization, their high invocation cost and runtime overhead make direct use of LLMs impractical for parsing massive real-time logs.

A noticeable fact about logs in real-world scenarios is their high degree of repetitiveness, with many logs being identical or only slightly differing in a few parameters. This is because they share the same log template. Therefore, parsing these repetitive logs is undoubtedly a waste of resources and reduces the efficiency of log parsers. Among the existing syntax-based log parsers, Drain [10] is popular for its high efficiency. It uses a prefix tree structure to group logs, and can quickly search and match templates when new logs arrive, avoiding duplication calculation. Inspired by Drain, we develop the **EffiParsing tree** to speed up the parsing process in SemanticLog.

As illustrated in Fig. 4(b), the EffiParsing tree adopts a prefix tree structure, building intermediate nodes based on the first token of a log and using a log template library as leaf nodes. However, different from Drain, which initially groups logs by length based on the assumption that logs with the same template have the same length, which actually often does not hold true due to variable lengths of log variables. This leads to logs with the same template having different lengths. Here, we employ a more direct and effective method to construct the parsing tree. For logs whose prefix token is a constant, we group them by their prefix tokens and use the first constant token as the key to establish the first layer of intermediate nodes. Then, we store their log templates in the leaf nodes of this node. For logs whose prefix token is a variable, we create a first-layer intermediate node with the key ''<*>'', then check the suffix token of the log. If it is a constant, we use this suffix token as the key to establish the second layer of intermediate nodes, and store their log templates in the leaf nodes of this node. If the suffix token is still a variable, we create a second-layer intermediate node with the key ''<*>'', storing the log template in this node's leaf nodes. This rule effectively reduces the number of

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

log templates stored in each leaf node, thereby lowering the cost of search and match.

x The EffiParsing tree is built and refined in sync with the inference process of SemanticLog. When a new log arrives, it first enters the EffiParsing tree for log template matching. If the search can proceed along the tree into a leaf node, the log is compared by similarity with each template in the current leaf node's template library as follows:

$$Sim = \frac{len(\mathcal{S}(Log) \cap \mathcal{S}(Template))}{len(\mathcal{S}(Log))} \tag{4}$$

where $\mathcal{S}$ denotes the function that splits the raw log into log tokens based on space. We select the three most similar (if available) log templates for regex matching with the log. If the matching is successful, we directly output this matched template along with the semantic labels of its parameters as the prediction result. If there is no successful template matching or the leaf node cannot be reached, the log is just ordinarily parsed. The EffiParsing tree is updated according to the prediction result, and the new log template and its corresponding parameter semantic labels are stored.

There is prior work exploring ways to accelerate LLM-based parsers. LILAC introduces the Adaptive Parsing Cache. However, the Adaptive Parsing Cache uses a separate leaf node for each log template, with each intermediate node representing a token. This design leads to a highly complex tree structure, particularly when dealing with complex log templates, resulting in very deep and unbalanced trees. Traversing such deep trees to match templates incurs considerable computational cost. In contrast, our EffiParsing tree employs a more direct and efficient approach. It utilizes a prefix tree structure that groups logs by their first token, with log templates in each group sharing a single leaf node. Combined with similarity matching, as detailed in Equ.(4), this improves template matching efficiency within each group. As a result, our tree structure maintains a shallow, fixed depth, making it far more efficient in handling large-scale, complex logs. Our experiments validate this point (See Sec. VII-C).

VI. Experimental Design

A. Research Questions

We conduct extensive experiments on the public benchmark LogHub-2.0 [35] to answer the following research questions:

RQ1: How effective is SemanticLog?
RQ2: How efficient is SemanticLog?
RQ3: How do the individual components contribute to SemanticLog?

B. Environment and Implementation

We conduct experiments on a GPU server equipped with Nvidia GeForce RTX3090 and CUDA11.3, and the model is implemented using python3.8 and Pytorch1.8.1. We choose the widely used LLaMA2-7B [22] as the base model to build LogLLM, and then fine-tune it using data from LogHub-2.0. In the fine-tuning stage, we use the standard parameter-efficient fine-tuning method LoRA [43] to fine-tune the model. We configure the LoRA parameters $lora_rank$, $lora_alpha$ and $lora_dropout$ to 12, 32, 0.1 respectively. We set the batch size to 8, the initial learning rate to 1e-4, and the number of training epochs to 10. During the inference parsing phase, we set the batch size to 100.

Most existing methods use the same source for training and testing, resulting in poor generalization and significant performance drops when encountering new log types. In contrast, we follow the UniParser [15] approach by using heterogeneous logs from different sources for training and testing, ensuring that log templates in the training and testing sets are mutually exclusive to verify the model's generalization capability. For LogHub-2.0, we divide the data by log source and conduct independent experiments on each source. Specifically, for a target log source, we train the model on a mixture of logs from the other 13 sources and test it on the target source, resulting in 14 independent training-testing trials. A recent study [44] has shown that language models can achieve good performance in log analysis tasks with only a small amount of training data. To reduce the burden of preparing training data, we sample only about 4.5% of the labeled logs for training and test the model on the entire test set. Experimental results show that even with minimal training data, our method demonstrates strong generalization and achieves state-of-the-art results. For all experiments that exhibit randomness, we repeat them five times and report the median results following previous work [24], [35] to avoid potential random bias.

C. Datasets and Baselines

1) Datasets: LogHub-2.0 [35], introduced by the LogPAI team, is the most popular log parsing dataset collection, containing a total of 50.4 million logs with approximately 3,500 log templates. The logs are divided into 14 datasets of varying sizes, from the smallest Proxifier dataset with 21,320 logs and 11 templates to the largest Thunderbird dataset with 16,601,745 logs and 1,241 templates. To evaluate semantic parsing performance on LogHub-2.0, we annotate it with semantic labels based on the semantic category system described in Sec. III.

2) Baselines: We compare our method, SemanticLog, with 11 state-of-the-art log parsers, including 10 general log parsers and 1 semantic log parser. The general parsers include syntax-based approaches (AEL [11], Drain [10], LenMa [12], Brain [45]), deep learning-based methods (UniParser [15], LogPPT [14]), and LLM-based parsers (LILAC [24], LLMParser [26], HELP [39], LUNAR [36]). For semantic log parsing, we compare our method with VALB [17], as LogPTR [33] and SemParser [16] are not open-source and lack implementation details. For the methods [12], [24], [36], [39], [45] that provide performance results on LogHub-2.0 in the literature, we use the data they reported. For those [10], [11], [14], [15], [17], [26] that do not provide performance results on LogHub-2.0 in the original papers, we rerun these methods on LogHub-2.0 and report the results.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

D. Metrics

Following prior studies [14], [15], [24], we apply three commonly used metrics for traditional log parsing evaluation, including Group Accuracy (GA), Parsing Accuracy (PA), and the F1 score of Template Accuracy (FTA).

For the evaluation of semantic log parsing, we introduce two additional novel metrics: Semantic Parsing Accuracy (SPA) and Semantic Parsing Accuracy Plus (SPA+).

Group Accuracy (GA): This metric measures the parser's ability to accurately cluster logs. It is calculated as the proportion of correctly grouped logs to the total number of logs. However, GA cannot measure whether the parser accurately distinguishes templates and parameters in logs.

Parsing Accuracy (PA): PA is a stricter metric than GA, assessing a model's ability to distinguish between templates and parameters in logs. A log is only deemed "correctly parsed" if each token is accurately identified. It is calculated as the ratio of correctly parsed logs to the total number of logs. Unlike GA, PA offers a more accurate reflection of a model's log parsing performance.

F1 score of Template Accuracy (FTA): This metric measures parsing quality through two metrics: Precision Template Accuracy (PTA) and Recall Template Accuracy (RTA). PTA calculates the proportion of correctly parsed templates among all identified ones, while RTA evaluates the ratio of correctly parsed templates over the total correct templates. FTA is the harmonic mean of them, it is a template-level metric.

Semantic Parsing Accuracy (SPA): We propose SPA specifically to evaluate semantic log parsing performance. We divide the process of judging whether a log has been correctly parsed into two steps. First, we check whether the model correctly distinguishes the log template and parameters, and then for each log parameter, we check whether the model's semantic prediction result contains the true semantic category of the parameter. If the prediction result of a log satisfies these two conditions, it is regarded as having been correctly parsed. SPA is calculated as the proportion of correctly parsed logs to the total number of logs. For the considerations described in Sec. IV, SPA is a reasonable indicator for evaluating semantic log parsing, balancing the rigor and practicality of the evaluation.

Semantic Parsing Accuracy Plus (SPA+): For stricter scenarios, we employ SPA+ to measure the model's performance in semantic log parsing. Similar to SPA, SPA+ is based on the model's ability to predict the semantic labels for parameters. However, SPA+ demands that the label of the highest probability in the model's output matches the true semantic category, and this must hold for all parameters in a log to be considered that the log is correctly parsed. So SPA+ is a more stringent metric than SPA, sacrificing some practicality to assess the precision of the model's semantic prediction more strictly.

VII. Experimental Results

A. RQ1: How Effective is SemanticLog?

For this research question, we delve into the effectiveness of SemanticLog in both traditional log parsing and semantic log parsing. We conduct comparisons on LogHub-2.0 against eleven popular SOTA log parsers. To conduct a comprehensive analysis, we utilize a rich set of evaluation metrics. Table II shows the experimental results on LogHub-2.0, where the best result for each metric is marked in **bold**, while the second-best result is <u>underlined</u>. Some metrics are "–" because these parsers cannot complete the parsing of the logs within a reasonable period (24 hours).

Experimental results show that SemanticLog achieves the best average performance across all five metrics (GA, PA, FTA, SPA, and SPA+), outperforming the SOTA log parsers, including the latest LLM-based parsers. Additionally, SemanticLog achieves the best or second-best PA and FTA results in 10 out of the 14 data sources, demonstrating its robustness.

Among the traditional log parsers, LUNAR performs the best, with an average GA of 93.3%, an average PA of 88.4%, and an average FTA of 81.6%. In comparison, SemanticLog achieves a comparable average GA, while improving PA and FTA by 5.2% and 2.8%, respectively. Note that LUNAR relies on online services provided by ChatGPT, whereas SemanticLog uses much smaller open-source LLMs. The superior performance of SemanticLog is attributed to its LogLLM module, which adapts the structure of LLMs specifically for log parsing, fully leveraging the pre-trained knowledge of LLMs. By treating log parsing as a token classification task, SemanticLog avoids errors caused by the redundancy or inconsistencies often found in LLM-generated outputs.

As for semantic log parsing, SemanticLog significantly outperforms VALB in both SPA and SPA+. SemanticLog achieves an average SPA of 83.2%, an 18.7% improvement over VALB, and an average SPA+ of 55.9%, 10.1% higher than VALB. This performance improvement is due to the effective collaboration between the LogLLM and SemPerception modules. After LogLLM extracts log feature representations, the SemPerception module maps these features into the log semantic space and performs accurate classification, greatly enhancing the performance of semantic log parsing.

To analyze the suboptimal performance of VALB on certain datasets, we take Hadoop and OpenSSH as examples.

1) Hadoop Dataset: For the log: `''Adding protocol org.apache.hadoop.mapreduce.v2.api.MRClientProtocolPB to the server''`

Ground truth (GT) template: `''Adding protocol <*> to the server''`

GT semantic template: `''Adding protocol <OBN> to the server''`

Here, `org.apache.hadoop.mapreduce.v2.api.MRClientProtocolPB` is a fully qualified class name, which belongs to Object Name (OBN). VALB predicts: `''Adding protocol <LOI> to the server''`, incorrectly classifying the parameter as a location (LOI). This is likely because the format `xxxx.xxxx.xxxx` resembles an IP address. The error reflects VALB's weak semantic understanding. Given that Hadoop logs frequently contain class names, this limitation significantly affects VALB's performance.

For the log: `''MapCompletionEvents request from attempt_1445087-`

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ACCURACY COMPARISON WITH SOTA LOG PARSERS ON THE LOGHUB-2.0 BENCHMARK

Type	Method	Metric	Apache	BGL	Hadoop	HDFS	HealthApp	HPC	Linux	Mac	OpenSSH	OpenStack	Proxifier	Spark	Thunderbird	Zookeeper	**Average**
Traditional Log Parsers																	
Syntax-based Log Parsers	**AEL**	GA	**100.0**	91.5	82.3	99.9	72.5	74.8	91.6	79.7	70.5	74.3	97.4	-	78.6	99.6	85.6
		PA	72.7	40.6	53.5	62.1	31.1	74.1	8.2	24.5	36.4	2.9	67.7	-	**78.6**	84.2	44.2
		FTA	51.7	16.5	5.8	56.2	0.3	13.6	21.7	20.5	33.3	16.5	41.7	-	3.5	46.5	25.2
	Drain	GA	**100.0**	91.9	92.1	99.9	86.2	79.3	68.6	76.1	70.7	75.2	69.2	88.8	83.1	99.4	84.3
		PA	72.7	40.7	54.1	62.1	31.2	72.1	11.1	35.7	58.6	2.9	68.8	39.4	21.6	84.3	46.8
		FTA	51.7	19.3	38.4	60.9	0.4	15.2	25.9	6.9	48.7	0.2	17.6	41.2	7.1	61.4	28.2
	LenMa	GA	99.3	-	79.6	99.9	-	79.3	80.6	70.1	74.8	85.1	50.4	-	-	85.7	80.0
		PA	3.1	-	5.2	13.7	-	64.2	3.5	12.1	14.2	1.9	49.5	-	-	68.3	24.0
		FTA	26.7	-	0.9	40.4	-	1.5	17.0	2.1	20.0	6.0	15.1	-	-	26.3	16.0
	Brain	GA	99.7	94.0	56.3	96.0	97.9	80.0	79.0	83.4	66.3	**100.0**	52.1	97.2	79.2	99.3	84.3
		PA	28.7	40.2	14.3	92.9	17.5	66.3	1.0	32.5	48.1	14.1	70.3	39.3	26.1	82.2	41.6
		FTA	46.7	19.7	20.0	62.1	33.9	21.3	27.5	29.4	34.5	29.2	73.7	1.0	27.4	60.1	35.4
Deep Learning-based Log Parsers	**LogPPT**	GA	78.6	24.5	48.3	72.1	99.8	78.2	20.5	54.4	27.7	53.4	98.9	47.6	56.4	96.7	61.2
		PA	94.8	93.8	66.6	94.3	**99.7**	**99.7**	12.8	39.0	65.4	40.6	**100.0**	95.2	40.1	84.5	73.6
		FTA	36.8	26.1	43.4	31.2	82.2	76.8	42.8	27.4	10.5	73.8	95.7	29.9	11.7	80.9	47.8
	UniParser	GA	94.8	91.8	69.1	**100.0**	46.1	77.7	28.5	73.7	27.5	**100.0**	50.9	85.4	57.9	98.8	71.6
		PA	94.2	94.9	88.9	94.8	81.7	94.1	16.4	68.8	28.9	51.6	63.4	79.5	65.4	98.8	73.0
		FTA	26.9	21.9	47.6	58.1	46.2	35.1	23.2	28.3	0.5	28.9	45.7	1.2	29.0	51.0	31.7
LLM-based Log Parsers	**LILAC**	GA	**100.0**	89.4	87.2	**100.0**	**100.0**	86.9	**97.1**	87.6	69.0	**100.0**	**100.0**	**100.0**	80.6	**100.0**	92.7
		PA	99.6	95.8	83.2	99.9	72.9	70.5	76.5	63.8	94.1	**100.0**	**100.0**	97.3	55.9	68.7	84.2
		FTA	**86.2**	74.6	77.9	94.6	87.3	80.0	**74.0**	55.3	86.5	**97.9**	**100.0**	75.9	57.2	86.8	81.0
	LLMParser	GA	45.2	-	57.6	-	74.2	93.6	19.1	54.3	33.5	93.2	**100.0**	-	-	75.0	64.6
		PA	99.1	-	72.0	-	85.9	94.1	57.1	54.0	69.1	92.6	**100.0**	-	-	80.8	80.5
		FTA	40.7	-	3.6	-	52.3	21.6	12.0	5.0	2.8	35.9	93.8	-	-	10.6	27.8
	HELP	GA	99.3	**99.9**	92.3	99.9	99.1	85.4	77.4	81.7	62.5	94.0	84.7	80.0	76.2	99.9	88.0
		PA	96.9	**99.6**	**90.8**	69.0	97.5	99.0	75.6	57.7	62.2	94.6	93.9	93.4	64.4	66.9	83.0
		FTA	75.4	**91.8**	79.2	63.0	84.2	**90.8**	62.5	56.5	68.6	82.5	69.6	78.6	**63.8**	87.8	75.3
	LUNAR	GA	**100.0**	94.9	**93.7**	**100.0**	99.8	86.4	94.8	77.0	78.0	**100.0**	**100.0**	97.5	85.6	99.2	**93.3**
		PA	**99.8**	98.2	86.0	**100.0**	98.2	99.3	85.8	53.1	72.2	98.0	**100.0**	99.6	62.6	85.2	88.4
		FTA	**86.2**	80.1	74.0	94.6	**88.0**	81.6	73.5	57.0	**90.0**	93.8	**100.0**	76.4	59.9	87.4	81.6
Semantic Log Parsers																	
Deep Learning-based Log Parsers	**VALB**	GA	97.0	82.5	28.0	94.8	65.3	84.8	13.7	63.9	27.7	53.4	1.0	72.2	48.1	83.9	58.3
		PA	97.4	96.1	49.0	99.4	93.2	94.5	86.8	78.8	90.7	49.6	86.5	99.7	53.6	89.9	83.2
		FTA	72.7	12.0	1.4	0.5	75.4	48.8	62.1	30.8	1.4	68.3	15.2	59.6	42.5	2.2	35.2
		SPA	89.0	**45.4**	26.1	99.4	35.2	93.8	84.4	52.2	89.7	49.1	51.2	59.5	43.1	85.4	64.5
		SPA+	54.8	19.4	17.7	69.2	24.4	75.5	**66.1**	31.4	38.7	37.3	50.1	**41.8**	32.5	83.1	45.8
LLM-based Log Parsers	**SemanticLog**	GA	97.9	96.0	92.6	**100.0**	94.1	**99.7**	57.9	**91.0**	**95.3**	**100.0**	93.0	99.8	**90.2**	99.3	**93.3**
		PA	97.2	94.0	90.6	**100.0**	96.2	**99.7**	**93.8**	**80.0**	**98.9**	99.2	86.9	**99.8**	75.1	**99.3**	**93.6**
		FTA	81.3	80.4	**81.3**	**98.7**	82.3	83.2	65.3	**90.9**	81.1	96.3	86.7	**99.8**	61.1	**93.6**	**84.4**
		SPA	**94.6**	44.0	**65.1**	**100.0**	**89.6**	**99.7**	**90.3**	**63.1**	**95.4**	**98.2**	**86.9**	**71.2**	**67.9**	**99.2**	**83.2**
		SPA+	**82.3**	**31.2**	**31.4**	**86.4**	**62.4**	**77.1**	27.6	**47.0**	**67.5**	**39.6**	**55.9**	27.3	**52.5**	**94.7**	**55.9**

491445_0006_r_000000_0. startIndex 0 maxEvents 10000''

GT template: ``MapCompletionEvents request from <*>. startIndex <*> maxEvents <*>''

GT semantic template: ``MapCompletionEvents request from <OID>. startIndex <OBO> maxEvents <OBA>''

Here, 0 is an index used to indicate the position in the event list, and it belongs to Object Order (OBO). VALB predicts: ``MapCompletionEvents request from <OID>. startIndex <OID> maxEvents <OBA>''. It misclassifies the second parameter 0 as Object ID instead of Object Order, likely due to limited contextual understanding — ignoring the keyword startIndex. This is the second most frequent template in Hadoop, so the error has a notable impact on overall performance.

2) OpenSSH Dataset: For the log: ``message repeated 5 times: [Failed password for root from 180.101.249.16 port 53496 ssh2]''

GT template: ``message repeated <*> times: [Failed password for <*> from <*> port <*> ssh2]''

GT semantic template: ``message repeated <OBA> times: [Failed password for <OBN> from <LOI> port <OID> ssh2]''

Here, 5 is a count indicating how many times the message was repeated, and it belongs to Object Amount (OBA). VALB predicts: ``message repeated <OID> times: [Failed password for <OBN> from <LOI> port <OID> ssh2]''. It misclassifies the count 5 as Object ID (OID) instead of Object Amount (OBA), again due to poor contextual reasoning—overlooking the word "times". This is the most frequent template in OpenSSH, so this error substantially degrades VALB's performance on this dataset.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE III
INFERENCE TIME COMPARISONS WITH SOTA PARSERS ON LOGHUB-2.0

Parser	Syntax-Based		Deep Learning-Based			LLM-Based				
	Drain	**AEL**	**UniParser**	**LogPPT**	**VALB**	**LLMParser**	**LILAC**	**HELP**	**LUNAR**	**SemanticLog**
Running time (s)	931	1793	2293	4097	4019	timeout	1685	16202	3578	1492

While VALB is a deep learning model whose performance naturally varies across datasets, a fully theoretical analysis is challenging. The above case studies suggest that weak semantic and contextual understanding are likely reasons for its poor performance on Hadoop and OpenSSH.

B. RQ2: How Efficient is SemanticLog?

Efficiency is another important performance measure of log parser. Existing deep learning-based log parsers often cannot be used in practice due to their low efficiency [24]. Only log parsers capable of handling large-scale logs can meet the requirements of modern cloud systems. For this research question, we evaluate the efficiency of SemanticLog on the LogHub-2.0 dataset by comparing its training and inference time with that of existing parsers. We reran these baseline methods under the same experimental conditions for a fair comparison.

1) Training Time: We compare the average training time of SemanticLog with three trainable methods, including LogPPT, VALB, and LLMParser. Since UniParser [15] is not open-source, we do not include it in the comparison. Table IV presents the training time comparison results on a server equipped with 8 RTX 3090 GPUs.

LogPPT and VALB are deep learning based methods built on models such as BERT and LSTM, without using LLMs, so their training is relatively fast. LogPPT adopts a few-shot training strategy, making its training time the shortest. In contrast, LLMParser and our method SemanticLog are based on open-source LLMs. Due to the large number of parameters in LLMs, their training is slower. Although LLMParser uses a few-shot training mode, its sampling process is very time-consuming, resulting in a long overall training time. Our method is slower than deep learning based approaches like LogPPT and VALB, but it is several times faster than LLMParser, which is also LLM-based. Considering that training is performed offline before deployment, and given the superior performance of SemanticLog, we believe that such training time is acceptable in practice.

2) Inference Time: We compare SemanticLog with three types of existing log parsers in terms of efficiency, including 2 syntax-based parsers, 3 deep learning-based parsers, and 4 LLM-based parsers. To reduce the impact of randomness and network latency fluctuations, especially for methods relying on OpenAI's online services [24], [36], [39], we conduct multiple evaluations for each method and report the average runtime. The results are presented in Table III. We can see that SemanticLog's running speed surpasses all the popular deep learning-based parsers, all LLM-based parsers, as well as the syntax-based parser AEL. Although SemanticLog is slower than Drain, it achieves significantly better performance and is the second-fastest log parser in the evaluation. This is attributed to the EffiParsing tree, which stores log templates and matches predictions using a carefully designed prefix tree structure. It can effectively reduce the number of LLM calls during large-scale log parsing, thus speeding up the process. Moreover, since SemanticLog runs locally on an open-source LLM, it is not impacted by network conditions, making its efficiency more stable than the methods relying on online LLM services.

TABLE IV
TRAINING TIME COMPARISONS WITH TRAINABLE METHODS ON LOGHUB-2.0

Model	Few-Shot Training	Training Time (h)
LogPPT	✓	0.17
VALB	×	0.36
LLMParser	✓	17.14
SemanticLog	×	2.12

TABLE V
ABLATION STUDY OF LOGLLM

	GA	**PA**	**FTA**	**SPA**	**SPA+**
LLaMA2-7B	0.3	0.1	0.3	0.01	0.01
Finetuned LLaMA2-7B	60.1	69.5	43.5	39.4	39.4
$LogLLM_{(LLaMA2-7B-based)}$	**93.3**	**93.6**	**84.4**	**83.2**	**55.9**

C. RQ3: How Do the Individual Components Contribute to SemanticLog?

For this research question, we investigate the contributions of different modules in SemanticLog by conducting ablation experiments on the LogHub-2.0 benchmark.

Contribution of LogLLM: LogLLM serves as the backbone of SemanticLog. It adapts the structure of open-source LLMs to log parsing. To validate its contribution, we compare SemanticLog, which utilizes LogLLM (based on LLaMA2-7B), with the original LLaMA2-7B-chat and a version of LLaMA2-7B-chat fine-tuned on LogHub-2.0. As presented in Table V, although LLaMA2 has vast pre-trained knowledge it cannot understand how to parse logs, leading to extremely low performance. Even after fine-tuning, its performance is still unsatisfactory. Since it can only predict a single label for log parameters, its SPA is numerically equal to its SPA+. SemanticLog surpasses the classic LLaMA2 model across various metrics. This indicates that with the same number of parameters and pre-trained weights, LogLLM performs better in log parsing, proving its effectiveness in handling log data.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ABLATION STUDY OF SEMPERCEPTION MODULE

	GA	PA	FTA	SPA	SPA+
SemanticLog$_{base}$	55.0	78.9	51.3	55.9	34.8
w/ *AveragePooling*	66.3	83.1	58.3	61.2	39.7
w/ *SemPerception module*	**93.3**	**93.6**	**84.4**	**83.2**	**55.9**

TABLE VII
EFFICIENCY COMPARISON BETWEEN ADAPTIVE PARSING CACHE AND EFFIPARSING TREE

	Running Time (s)
SemanticLog$_{base}$	52136
w/ *Adaptive parsing cache*	1923
w/ *EffiParsing tree*	**1492**

TABLE VIII
OVERLAPPING RATIOS OF LOG TEMPLATES IN THE 14 SUBSETS OF LOGHUB-2.0

Subset	Overlapping Ratio
Apache	24.16%
BGL	19.07%
Hadoop	6.01%
HDFS	17.81%
HealthApp	2.39%
HPC	3.02%
Linux	9.82%
Mac	24.33%
OpenSSH	12.21%
OpenStack	21.27%
Proxifier	22.58%
Spark	3.55%
Thunderbird	17.45%
Zookeeper	3.63%
Average	13.38%

Contribution of the SemPerception module: The SemPerception module integrates subword features of log tokens to fully perceive the semantics of logs. To assess its effect, we conduct three experiments: 1) SemanticLog$_{base}$ where the SemPerception module is removed from SemanticLog, and the model performs semantic classification directly based on the log features extracted by LogLLM; 2) Based on SemanticLog$_{base}$, we use AveragePooling to aggregate the features of different subwords in the log tokens and perform semantic classification according to the aggregated features; 3) On the basis of SemanticLog$_{base}$, we add the SemPerception module to aggregate the log token features and map them into the semantic space for semantic classification. Results are presented in Table VI. We can see that the addition of the SemPerception module leads to a significant performance improvement. This is because log parameter tokens are generally long, and the SemPerception module can fully retain the original information of log tokens and perceive the semantics of logs by multi-head cross attention, accurately determining the categories of log tokens.

Contribution of the EffiParsing tree: The EffiParsing tree uses a prefix tree structure to store known templates and match predictions for new logs. Since previous work also proposes acceleration modules, here we conduct three experiments to evaluate the actual effectiveness of the EffiParsing tree: 1) SemanticLog$_{base}$ where we remove the EffiParsing tree module from the SemanticLog model, and use it to parses logs one by one; 2) Accelerating SemanticLog$_{base}$ with the adaptive parsing cache proposed in LILAC [24]; and 3) accelerating SemanticLog$_{base}$ with the EffiParsing tree, i.e., the complete SemanticLog model. The experimental results are presented in Table VII. We can see that the EffiParsing tree greatly improves the inference speed of SemanticLog, enabling it to handle large-scale logs. Without the EffiParsing tree, SemanticLog would require LLM to parse each new log, which would result in substantial time costs. Compared to the adaptive parsing cache, the EffiParsing tree demonstrates more significant acceleration on the complex LogHub-2.0 benchmark, thanks to its more direct and efficient tree structure.

Indeed, different templates may share identical prefix/suffix tokens due to EffiParsing tree's fixed-height design. This causes multiple templates to be mapped to the same node, increasing similarity computation cost. To quantify this impact, Table VIII shows the ratios of templates sharing prefix/suffix tokens in all 14 LogHub-2.0 subsets, with an average of 13.38%. Given that this ratio is relatively low and similarity computation can be accelerated via parallel processing, the overall computational overhead and runtime remain acceptable. Nevertheless, we leave the improvement of the EffiParsing tree for future work.

In summary, these ablation studies above prove the effectiveness of each module in SemanticLog. For the semantic log parsing task, they respectively strengthen the performance of LLM in extracting log features, perceiving log semantics, and accelerating parsing speed, making SemanticLog a semantic log parser that balances both effectiveness and efficiency.

VIII. DISCUSSIONS

A. Enhancing Model Performance through Few-Shot Tuning

SemanticLog has good generalization, enabling it to effectively do semantic parsing of heterogeneous logs from different sources. To explore the upper limit of SemanticLog's performance, we fine-tune it with few-shot annotated logs from the target log source. Following the setup of previous work [14], [15], we conduct experiments on the Mac dataset of LogHub-2.0, as the complexity of log templates in the Mac dataset is high (626 log templates), making it a good representation of real-world scenarios. As shown in Table IX, compared to the base$_{(0-shot)}$ model following the original setup in Sec. VI-B, SemanticLog achieves further performance improvement on the target log dataset with just a few annotated logs for fine-tuning.

B. Integration of SemanticLog With Different LLMs

As a key module of SemanticLog, LogLLM adapts LLMs to semantic log parsing by removing the causal mask. This

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
FEW-SHOT MODEL FINE-TUNING ON THE MAC DATASET

Target Samples	GA	PA	FTA	SPA	SPA+
$\text{base}_{(0-shot)}$	91.0	80.0	90.9	63.1	47.0
8-shot	90.6	82.5	92.2	77.2	55.7
16-shot	92.8	85.4	93.0	80.3	57.7

TABLE X
PERFORMANCE COMPARISON OF SEMANTICLOG WITH DIFFERENT LLMS ON THE MAC DATASET

Base LLMs	GA	PA	FTA	SPA	SPA+
LLaMA2-7B	91.0	80.0	90.9	63.1	47.0
LLaMA3.1-8B	87.1	86.8	90.8	69.3	54.3

TABLE XI
LOG ANOMALY DETECTION PERFORMANCE COMPARISON BETWEEN DIFFERENT TRAINING STRATEGIES. *BASELINE*: USING REGULAR LOG TEMPLATES; *SEM. CAT.* [1]: USING LOG TEMPLATES WITH 10 COARSE-GRAINED SEMANTIC CATEGORIES; *SEM. CAT.*[2]: USING LOG TEMPLATES WITH 16 FINE-GRAINED SEMANTIC CATEGORIES

Method	Dataset	Training Strategy	Precision	Recall	F1 Score
LogFormer	BGL	*Baseline*	92.2%	84.4%	88.1%
		Sem. Cat.[1]	91.9%	85.1%	88.4%
		Sem. Cat.[2]	**93.4%**	**87.1%**	**90.1%**
	HDFS	*Baseline*	95.6%	86.3%	90.7%
		Sem. Cat.[1]	**97.2%**	86.9%	91.8%
		Sem. Cat.[2]	95.5%	**91.0%**	**93.2%**
PLELog	BGL	*Baseline*	89.8%	91.2%	90.5%
		Sem. Cat.[1]	82.9%	**99.8%**	90.6%
		Sem. Cat.[2]	**89.9%**	**99.8%**	**94.5%**
	HDFS	*Baseline*	98.6%	83.3%	90.4%
		Sem. Cat.[1]	97.0%	87.3%	91.9%
		Sem. Cat.[2]	**98.7%**	**88.2%**	**93.1%**

allows LogLLM to be independent of any specific LLM, enabling it to be built on top of any open-source LLMs. In our implementation, we use LLaMA2-7B as the base model for building LogLLM due to its high performance and wide adoption. Additionally, we construct LogLLM using the more advanced LLaMA3.1-8B and perform a performance comparison between the two models on the complex Mac dataset. As shown in Table X, both LLMs perform exceptionally well. LLaMA3.1-8B gets improvements over LLaMA2-7B in PA, SPA, and SPA+, which can be attributed to its larger number of parameters and more comprehensive pre-training data, granting LLaMA3.1 stronger semantic understanding capability. These results show that SemanticLog can work with different LLMs while maintaining excellent performance.

C. The Effect of Semantic Log Parsing on Downstream Log Analysis Tasks

VALB [17] has demonstrated that log parameters contain valuable semantic information, and semantic log parsers can identify the semantics of these parameters, providing better support for downstream log analysis tasks than traditional (non-semantic) parsers. However, the 10 semantic categories defined in VALB are relatively coarse, making it difficult to capture the fine semantics in real-world logs. Therefore, in Sec. III, we define 16 fine-grained log semantic categories to address this limitation. To further investigate the effect of log semantics on downstream tasks, we use log anomaly detection as a case study to demonstrate the benefits of log semantics and the advantage of fine-grained semantic categories in enhancing downstream task performance.

In our experiments, we use the log anomaly detection dataset from LogPAI, featuring data from BGL and HDFS sources, and adopt the advanced deep model LogFormer [46] and PLELog [47], [48]. LogFormer is a transformer-based log anomaly detection framework, that achieves good generalization power through a two-stage training process involving pre-training and adapter-based tuning. PLELog is a semi-supervised log anomaly detection method that uses probabilistic label estimation and an attention-based GRU network. We consider three strategies for comparison: 1) Training and evaluating the log anomaly detection models with regular log templates that do not contain semantic labels, serving as the baseline. 2) Replacing the wildcard '<*>' in regular log templates with 10 coarse-grained semantic categories defined in VALB, then using these data to train and evaluate LogFormer. 3) Replacing the wildcard '<*>' in regular log templates with 16 fine-grained semantic categories defined in our paper, then using these data to train and evaluate log anomaly detection models. The results are presented in Table XI.

From Table XI, we can see that models trained with log semantics achieve higher F1 Scores, with the best performance being observed on models using fine-grained semantic categories. Specifically, LogFormer trained with 16 fine-grained semantic categories (defined in this paper) outperforms those trained with 10 coarse-grained semantic categories (defined in VALB) by 1.7% and 1.4% on the two datasets, respectively, and exceeded those trained with regular templates by 2% and 2.5%. PLELog trained with 16 fine-grained semantic categories achieves 3.9% and 1.2% improvements over that trained with 10 coarse-grained categories on the two datasets, respectively, and also outperforms the version trained with regular templates by 4% and 2.7%. The primary reason for this performance improvement is the increase in Recall, indicating that log semantic information aids in better understanding of logs, allowing the models to detect subtle anomalies more effectively. Fine-grained semantic categories provide more detailed semantic information, which significantly boosts anomaly detection performance.

D. Dependency of Existing LLM-Based Parsers on ChatGPT

Most existing LLM-based parsers rely on OpenAI's online ChatGPT APIs. While these methods benefit from ChatGPT's outstanding capabilities and achieve good results, they also raise serious concerns about data privacy. Due to the sensitive nature of log data, uploading logs to external servers is not allowed in most real-world applications.

A straightforward solution is to replace ChatGPT with an open-source LLM. We conduct experiments on LILAC, a representative LLM-based parser, replacing its backbone with LLaMA-2-7B-chat fine-tuned on log parsing data (to ensure a

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE XII
PERFORMANCE OF LILAC WITH DIFFERENT LLMS ON LOGHUB-2.0

Model Variant	GA	PA	FTA
LILAC$_{-ChatGPT}$	92.7	84.2	81.0
LILAC$_{-LLaMA2-7B}$	70.4	57.6	40.8

fair comparison with our method). As presented in Table XII, LILAC's performance drops significantly, with GA decreasing 22.3%, PA decreasing 26.6%, and FTA (the strictest metric) decreasing 40.2%. This shows LILAC's heavy dependence on ChatGPT, and that simply replacing it with an open-source model is not effective. Admittedly, using a larger open-source model could improve performance. For example, by LLaMA 3.1, we can achieve ChatGPT's performance only at the 70B scale [49], but such models require substantial hardware resources and high running costs.

The poor performance of smaller open-source LLMs (e.g., 7B-scale) stems from their limited parameter capacity, which weakens text generation and instruction-following abilities despite rich pre-training knowledge. Moreover, the causal mask in standard LLMs restricts bidirectional context flow, making it difficult to capture full semantic dependencies in log messages, which are critical for accurate parsing. As a result, even fine-tuning fails to yield satisfactory results. To overcome this, SemanticLog removes the causal mask and shifts to a discriminative architecture, enabling full context awareness and unlocking the potential of small LLMs. This key design choice allows SemanticLog to achieve SOTA performance while preserving data privacy and avoiding reliance on large, costly models.

E. Generality and Extensibility of the Log Semantic System

In Sec. III, we introduce our log semantic system, designed by combining our team engineers' expertise and the semantic patterns from the LogHub-2.0 dataset. Its generality has been validated on real-world business logs.

Our principle is to build a concise semantic category system. Hence, we avoid defining overly fine-grained semantic categories. However, certain downstream tasks may require finer distinctions, such as between Thread ID and Process ID, or Function Name and Parameter Name. These can be added as subcategories to form a hierarchical semantic system. For example, Thread ID and Process ID can be treated as subcategories of Object ID. The original semantic classification task in SemanticLog then becomes a hierarchical classification task, a well-studied problem in machine learning area. Solutions include flattening the hierarchy or using hierarchical classifiers with constraint-based losses. This extension can be easily implemented in SemanticLog by replacing the default linear classifier with a hierarchical classifier, without modifying the rest of the model.

IX. THREATS TO VALIDITY

Data quality. Large-scale datasets contain a small number of errors. To reduce the threat, we use the latest benchmark LogHub-2.0 as the evaluation dataset, which has been corrected manually [3], [14], [50].

Training effort. SemanticLog is an LLM-based log parser that requires the use of GPU for training and inference. In order to lower the application threshold, we optimize the model so that SemanticLog can run on the server with only a single RTX3090 GPU. Due to its good generalization, SemanticLog needs only to be trained once to parse heterogeneous logs from different sources without repeated training.

Configuration sensitivity. The performance of log parsers is greatly affected by the configuration, such as token delimiters and hyperparameters. Frequently adjusting these configurations leads to significant overhead. To reduce the threat, we use only spaces as delimiters and have no additional hyperparameters. Experiments show that SemanticLog achieves the best performance without the need for complex configurations.

X. RELATED WORK

A. Log Parsing

Log parsing is a fundamental step in automated log analysis that converts unstructured or semi-structured log messages into structured data. Existing log parsing techniques can be categorized into traditional (or non-semantic) log parsing and semantic log parsing, based on their parsing objectives. Traditional log parsing focuses on identifying static log templates and dynamic log parameters from raw logs. In contrast, semantic log parsing is built upon traditional log parsing and goes further to identify the semantic categories of log parameters.

1) Traditional Log Parsing: Existing traditional log parsers can be broadly categorized into unsupervised log parsers, supervised log parsers, and LLM-based log parsers.

Unsupervised log parsers [4], [9], [10], [11], [12], [13] typically use techniques such as frequent pattern mining, log clustering, and heuristic methods to identify log templates. However, these approaches often lack a fundamental understanding of the semantic information in logs, and their performance heavily depends on predefined rules and hyperparameters. These methods struggle with evolving log formats, requiring frequent hyperparameter tuning and ongoing maintenance.

Supervised log parsers [14], [15], [16], [17] rely on natural language processing (NLP) techniques. Some of these methods [15], [16], [17] treat log parsing as a token classification task, while LogPPT [14] fine-tunes a pre-trained language model for log parsing. Despite these advancements, their performance is highly dependent on the quality and diversity of training samples, which leads to poor generalization and suboptimal results when encountering previously unseen logs. Moreover, these methods are generally slower than traditional parsers in terms of parsing speed.

LLM-based log parsers [23], [24], [25], [26], [36], [39] represent a new direction. Le et al. [23] are among the first to evaluate ChatGPT's potential for log parsing. Some other methods such as DivLog [25], LILAC [24], and HELP [39], improve log parsing performance by incorporating similar log examples and carefully designed prompts. LUNAR [36] achieves example-free log parsing by combining log clustering with contrastive

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
KEY DIFFERENCES BETWEEN SEMANTICLOG AND EXISTING LLM-BASED PARSERS

	LILAC	LUNAR	HELP	LLMParser	SemanticLog
Open-source LLM				✓	✓
ChatGPT-dependent	✓	✓	✓		
Inference Acceleration	✓	✓			✓
Customized LLM					✓
Semantic Parsing					✓

analysis. However, these methods often work like a black-box due to their reliance on proprietary and non-transparent APIs, such as ChatGPT. The lack of open-source access and the unpredictability of version updates introduce performance variability. The sensitivity of log data also incurs practical challenges, as logs frequently contain private business information that requires local processing rather than online services. This limitation prevents these methods from being widely applied in real-world scenarios. LLMParser [26], which fine-tunes open-source LLMs for log parsing, avoids the privacy issue but suffers from limited performance and slow processing speed.

2) Semantic Log Parsing: VALB [17] pioneers semantic log parsing by defining it as a sequence labeling task, utilizing a BiLSTM-CNN-CRF architecture to classify the semantic roles of individual log tokens. SemParser [16] introduces a holistic framework that integrates semantic mining and joint parsing, leveraging domain knowledge to enhance semantic understanding. LogPTR [33] adopts pointer networks to develop an end-to-end semantic log parsing approach. However, these methods exhibit notable limitations. Their semantic log parsing performance significantly lags behind that of traditional log parsing, reflecting their weak capability to represent the semantic information of log categories. Moreover, as logs evolve and the data distribution of target logs diverges from that of the training logs, their performance deteriorates considerably.

B. Large Language Models

Evolving from the pre-trained language models, LLMs are based on self-supervised learning and use large-scale corpora to understand natural language texts. While retaining similar model architectures and pre-training tasks, simply expanding the size (e.g. from 1.5B GPT-2 [19] to 175B GPT-3 [20]) can endow LLMs with emergent ability to solve a series of complex tasks. LLaMA [21], [22], [49], released by Meta AI, is one of the most popular open-source LLMs. After fine-tuning, LLaMA is very suitable for use as a locally-run LLM. The extensive pre-trained knowledge of LLMs provides them with strong generalization, which can play an important role in log analysis [26], code generation [51], and other tasks in software engineering.

C. Key Differences Between SemanticLog and Existing LLM-based Parsers

Table XIII presents the key differences between SemanticLog and existing LLM-based log parsers from five perspectives: 1) whether an open-source LLM is used; 2) Whether the parser depends on ChatGPT; 3) Whether inference is accelerated; 4) Whether a customized LLM is used; 5) Whether the parser supports semantic log parsing. SemanticLog is the first to apply LLMs to semantic log parsing. Unlike prior works that rely on large online LLM services or simple fine-tuning of open-source LLMs, SemanticLog introduces a customized design tailored to the semantic log parsing task. It features three core components, including LogLLM, SemPerception Module, and EffiParsing Tree, which enhance small open-source LLMs in feature extraction, semantic understanding, and inference efficiency. With this design, SemanticLog achieves SOTA performance while preserving data privacy, outperforming existing methods dependent on online LLM APIs.

XI. CONCLUSION

This paper presents SemanticLog, a novel LLM-based semantic log parser. By adapting a specialized LLM structure for log parsing, SemanticLog exhibits strong generalization, effectively parsing heterogeneous logs from different sources. In SemanticLog, the SemPerception module enhances its ability to perceive log semantics, while the EffiParsing tree ensures its efficiency in parsing large-scale logs. Extensive experiments demonstrate that SemanticLog balances effectiveness and efficiency, and protects log privacy, establishing it as a SOTA semantic log parser with significant application potential.

XII. DATA AVAILABILITY

We have released our manual annotations for LogHub-2.0 [35] on FigShare, including corrected general templates and semantic templates for all 14 subsets. For the raw logs, please refer to the original LogHub-2.0 repository. We have also released the SemanticLog source code. For details, please see the README.md file in the repository. Project link: https://figshare.com/articles/dataset/SemanticLog/30178846.

ACKNOWLEDGEMENTS

The computations in this research were performed using the CFFF platform of Fudan University.

REFERENCES

[1] X. Wang et al., "SPINE: A scalable log parser with feedback guidance," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 1198–1208.
[2] X. Zhang et al., "Onion: Identifying incident-indicating logs for cloud systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1253–1263.
[3] J. Zhu et al., "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., Softw. Eng. in Pract. (ICSE-SEIP),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 121–130.
[4] H. Dai, H. Li, C.-S. Chen, W. Shang, and T.-H. Chen, "Logram: Efficient log parsing using n n-gram dictionaries," *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 879–892, Mar. 2022.
[5] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
[6] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 807–817.
[7] D. Shin, Z. A. Khan, D. Bianculli, and L. Briand, "A theoretical framework for understanding the relationship between log parsing and

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.

anomaly detection," in *Proc. Int. Conf. Runtime Verification*, New York, NY, USA: Springer-Verlag, 2021, pp. 277–287.

[8] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22nd Symp. Oper. Syst. Princ.*, 2009, pp. 117–132.

[9] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *Proc. IEEE 16th Int. Conf. Data Mining (ICDM),* Piscataway, NJ, USA: IEEE Press, 2016, pp. 859–864.

[10] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 33–40.

[11] Z. M. Jiang, A. E. Hassan, G. Hamann, and P. Flora, "An automated approach for abstracting execution logs to execution events," *J. Softw. Maintenance Evol., Res. Pract.*, vol. 20, no. 4, pp. 249–267, 2008.

[12] K. Shima, "Length matters: Clustering system log messages using length of words," 2016, *arXiv:1611.03213*.

[13] A. A. Makanju, A. N. Zincir-Heywood, and E. E. Milios, "Clustering event logs using iterative partitioning," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2009, pp. 1255–1264.

[14] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 2438–2449.

[15] Y. Liu et al., "UniParser: A unified log parser for heterogeneous log data," in *Proc. ACM Web Conf.*, 2022, pp. 1893–1901.

[16] Y. Huo, Y. Su, C. Lee, and M. R. Lyu, "SemParser: A semantic parser for log analytics," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 881–893.

[17] Z. Li et al., "Did we miss something important? Studying and exploring variable-aware log abstraction," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 830–842.

[18] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.

[19] A. Radford et al., "Improving language understanding by generative pre-training," 2018.

[20] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[21] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[22] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[23] V.-H. Le and H. Zhang, "Log parsing: How far can ChatGPT go?" in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 1699–1704.

[24] Z. Jiang et al., "LILAC: Log parsing using LLMs with adaptive parsing cache," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 137–160, 2024.

[25] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "DivLog: Log parsing with prompt enhanced in-context learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE),* Washington, DC, USA: IEEE Computer Society, 2024, pp. 983–983.

[26] Z. Ma, A. R. Chen, D. J. Kim, T.-H. Chen, and S. Wang, "LLMParser: An exploratory study on using large language models for log parsing," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE),* Washington, DC, USA: IEEE Computer Society, 2024, pp. 883–883.

[27] S. Lu, B. Rao, X. Wei, B. Tak, L. Wang, and L. Wang, "Log-based abnormal task detection and root cause analysis for spark," in *Proc. IEEE Int. Conf. Web Services (ICWS),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 389–396.

[28] N. Gurumdimma, A. Jhumka, M. Liakata, E. Chuah, and J. Browne, "Crude: Combining resource usage data and error logs for accurate error detection in large-scale distributed systems," in *Proc. IEEE 35th Symp. Reliable Distrib. Syst. (SRDS),* Piscataway, NJ, USA: IEEE Press, 2016, pp. 51–60.

[29] P. Notaro, S. Haeri, J. Cardoso, and M. Gerndt, "LogRule: Efficient structured log mining for root cause analysis," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4231–4243, Dec. 2023.

[30] S. Locke, H. Li, T.-H. P. Chen, W. Shang, and W. Liu, "LogAssist: Assisting log analysis through log summarization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3227–3241, Sep. 2022.

[31] W. Meng et al., "LogSummary: Unstructured log summarization for software systems," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3803–3815, Sep. 2023.

[32] W. Meng et al., "Summarizing unstructured logs in online services," 2020, *arXiv:2012.08938*.

[33] Y. Wu et al., "LogPTR: Variable-aware log parsing with pointer network," 2024, *arXiv:2401.05986*.

[34] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30.

[35] Z. Jiang et al., "A large-scale benchmark for log parsing," 2023, *arXiv:2308.10828*.

[36] J. Huang, Z. Jiang, Z. Chen, and M. Lyu, "No more labelled examples? An unsupervised log parser with LLMs," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 2406–2429, 2025.

[37] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367.

[38] H. Liu et al., "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 1950–1965.

[39] A. Xu and A. Gau, "Help: Hierarchical embeddings-based log parsing," 2024, *arXiv:2408.08300*.

[40] Q. Dong et al., "A survey on in-context learning," 2022, *arXiv:2301.00234*.

[41] Z. Du et al., "GLM: General language model pretraining with autoregressive blank infilling," 2021, *arXiv:2103.10360*.

[42] Z. Li et al., "Label supervised LLaMA finetuning," 2023, *arXiv:2310.01208*.

[43] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

[44] L. Ma et al., "KnowLog: Knowledge enhanced pre-trained language model for log understanding," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[45] S. Yu, P. He, N. Chen, and Y. Wu, "Brain: Log parsing with bidirectional parallel tree," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3224–3237, Sep./Oct. 2023.

[46] H. Guo et al., "LogFormer: A pre-train and tuning pipeline for log anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 1, pp. 135–143.

[47] L. Yang et al., "PLELog: Semi-supervised log-based anomaly detection via probabilistic label estimation," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Companion Proc. (ICSE-Companion),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 230–231.

[48] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367.

[49] A. Dubey et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.

[50] Z. A. Khan, D. Shin, D. Bianculli, and L. Briand, "Guidelines for assessing the accuracy of log message template identification techniques," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1095–1106.

[51] R. Jain, N. Gervasoni, M. Ndhlovu, and S. Rawat, "A code centric evaluation of C/C++ vulnerability datasets for deep learning based vulnerability detection techniques," in *Proc. 16th Innov. Softw. Eng. Conf.*, 2023, pp. 1–10.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on March 21,2026 at 09:19:04 UTC from IEEE Xplore. Restrictions apply.